

Bubble Sort Membandingkan dua elemen berdekatan dan menukarnya jika urutan salah. Elemen terbesar "mengambang" ke akhir setiap iterasi seperti gelembung naik ke permukaan

### Pseudo Code

```
PROCEDURE BubbleSort(A, n):
  FOR i ← 0 TO n-1:
    swapped ← false
    FOR j ← 0 TO n-i-2:
      IF A[j] > A[j+1]:
        SWAP A[j] ↔ A[j+1]
        swapped ← true
    IF swapped = false:
      BREAK ← sudah terurut
  RETURN A
```

### Implementasi Python

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr
data = [64, 34, 25, 12, 22, 11, 90]
print(bubble_sort(data))

[11, 12, 22, 25, 34, 64, 90]
```

Analisis Kompleksitas Best Case  $O(n)$  Average Case  $O(n^2)$  Worst Case  $O(n^2)$  Space (Aux)  $O(1)$  Worst case terjadi ketika array terurut terbalik. Best case  $O(n)$  dicapai dengan optimasi flag swapped algoritma berhenti lebih awal jika tidak ada pertukaran.

Selection Sort Mencari elemen minimum dari sisa array yang belum terurut, lalu menukarnya ke posisi yang benar. Memilih elemen secara berulang seperti memilih kartu terkecil dari tangan.

## Pseudo Code

```

PROCEDURE SelectionSort(A, n):
  FOR i ← 0 TO n-2:
    min_idx ← i
    FOR j ← i+1 TO n-1:
      IF A[j] < A[min_idx]:
        min_idx ← j
    IF min_idx ≠ i:
      SWAP A[i] ↔ A[min_idx]
  RETURN A

```

## Implementasi Python

```

def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
data = [64, 25, 12, 22, 11]
print(selection_sort(data))

```

```
[11, 12, 22, 25, 64]
```

Analisis Kompleksitas Best Case  $O(n^2)$  Average Case  $O(n^2)$  Worst Case  $O(n^2)$  Space (Aux)  $O(1)$  Konsisten  $O(n^2)$  di semua kasus karena selalu melakukan perbandingan penuh. Keunggulannya: jumlah swap minimum hanya  $O(n)$  swap, cocok ketika operasi tulis ke memori mahal.

Merge Sort Menggunakan strategi divide and conquer — membagi array menjadi dua bagian, mengurutkan masing-masing secara rekursif, lalu menggabungkannya. Konsisten dan dapat diprediksi.

## Pseudo Code

```

PROCEDURE MergeSort(A, left, right):
  IF left < right:
    mid ← (left + right) / 2

```

```

MergeSort(A, left, mid)
MergeSort(A, mid+1, right)
Merge(A, left, mid, right)

PROCEDURE Merge(A, left, mid, right):
  L ← A[left..mid], R ← A[mid+1..right]
  i ← 0, j ← 0, k ← left
  WHILE i < len(L) AND j < len(R):
    IF L[i] ≤ R[j]: A[k] ← L[i]; i++
    ELSE:          A[k] ← R[j]; j++
    k++
  Salin sisa L atau R ke A

```

## Implementasi Python

```

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

data = [38, 27, 43, 3, 9, 82, 10]
print(merge_sort(data))

[3, 9, 10, 27, 38, 43, 82]

```

Analisis Kompleksitas Best Case  $O(n \log n)$  Average Case  $O(n \log n)$  Worst Case  $O(n \log n)$  Space (Aux)  $O(n)$  Selalu  $O(n \log n)$  paling konsisten di antara keempat algoritma. Kelemahan utama: membutuhkan memori tambahan  $O(n)$ . Sangat cocok untuk linked list dan external sorting (data di disk).

Quick Sort Memilih pivot, mempartisi array agar elemen lebih kecil di kiri dan lebih besar di kanan, lalu rekursif ke kedua sisi. Rata-rata paling cepat dalam praktik.

### Pseudo Code

```

PROCEDURE QuickSort(A, low, high):
  IF low < high:
    p ← Partition(A, low, high)
    QuickSort(A, low, p-1)
    QuickSort(A, p+1, high)

PROCEDURE Partition(A, low, high):
  pivot ← A[high]
  i ← low - 1
  FOR j ← low TO high-1:
    IF A[j] ≤ pivot:
      i++
      SWAP A[i] ↔ A[j]
  SWAP A[i+1] ↔ A[high]
  RETURN i + 1

```

### Implementasi Python

```

def quick_sort(arr, low=0, high=None):
    if high is None:
        high = len(arr) - 1
    if low < high:
        p = partition(arr, low, high)
        quick_sort(arr, low, p - 1)
        quick_sort(arr, p + 1, high)
    return arr

def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

data = [10, 80, 30, 90, 40, 50, 70]
print(quick_sort(data))

```

[10, 30, 40, 50, 70, 80, 90]

Analisis Kompleksitas Best Case  $O(n \log n)$  Average Case  $O(n \log n)$  Worst Case  $O(n^2)$  Space (Aux)  $O(\log n)$  Worst case  $O(n^2)$  terjadi bila pivot selalu elemen terkecil/terbesar (array sudah terurut). Dalam praktik, Quick Sort seringkali lebih cepat dari Merge Sort karena cache-friendly dan overhead kecil. Diatasi dengan random pivot atau median of three.